GIET POLYTECHNIC COLLEGE

Artificial Intelligence (AI) Department | Batch 2024-2027

EMS

Employee Management System

Complete Technical Documentation

Industrial Training Project | Team T-1

Team Lead	Teja (Roll No. 2401)
Team Members	Ashok | Charan | Kula Shekhar | Jahnavi | Afsar
Publisher ID	GIETP-AI-IT-EMS-T1-2026
Application ID	in.gietpoly.ems
Release / Version	October 2026 | 1.0.0

Repository-grounded submission artifact generated on 11 October 2026

Source of truth: current repository docs, application code, SQL migrations, tests, and shared packages.

Document status. Document status and verification basis

A submission-ready reference with explicit evidence labels.

Control	Value
Artifact	EMS_Team_T1_Complete_Technical_Documentation.pdf
Project	Employee Management System (EMS) v1.0.0
Institution	GIET Polytechnic College - Artificial Intelligence Department
Team	T-1; Teja is documented as Team Lead
Generated	11 October 2026
Repository	C:\Users\TEJA\Documents\Employee Management System - EMS

Verification disclaimer

This PDF is a source-based technical documentation artifact. It does not by itself prove a live deployment, production availability, measured latency, or native mobile/desktop packaging. Statements are labeled as documented, observed, derived, implemented, foundation, pending, or needs verification. No live application screenshots are included because no repository-sourced live screenshots were supplied.

Evidence labels

Label	Meaning
Documented	Claim stated in a repository document; may describe a release baseline.
Observed	Result of a command executed for this PDF on the current working tree.
Derived	Count or inventory computed from current repository files.
Implemented	Feature has corresponding current code/schema/routes/tests; runtime gate may still need verification.
Architecture foundation	Shared scaffolding exists, but native platform delivery is not verified.
Pending / needs verification	The repository explicitly leaves the item unexecuted, unmeasured, or future.

Checks executed for this document

Check	Current observed result	Interpretation
pnpm test	8 suites; 103 passed; 0 failed	Observed pass on current working tree.
pnpm typecheck	Failed in apps/web/lib/auth/session.ts:387	Context activeOrganization nullability mismatch.
pnpm lint	Failed in auth/access.ts, auth/actions.ts, auth/session.ts	Boolean-literal comparison errors; two unused-disable warnings.
pnpm --filter @ems/web build	Compiled then failed type validation	Same session-context type error; no successful build claim.
pnpm db:test	Not executed	DEVELOPMENT_STATUS says Docker/Supabase availability is required.

Table of Contents

Section	Page
Document status. Document status and verification basis	2
1. Introduction and project context	4
2. Existing system versus proposed EMS	5
3. Software requirements specification	6
4. Current architecture and monorepo topology	7
5. Request, authentication and data flow	9
6. Database design and entity inventory	10
7. API and server operations	12
8. Security, RLS and threat controls	13
9. Roles, portals and permission coverage	14
9A. Full 22-permission RBAC matrix	16
10. Feature inventory and implementation status	18
11. Module workflows and algorithms	20
12. UI/UX and screen inventory	21
13. Team, lead and module ownership	22
14. Testing and actual-versus-planned status	24
15. Installation and deployment	26
16. Limitations and future scope	27
17. Conclusion	28
18. Viva voce and Q&A appendix	29

Reading guide

Page numbers are generated from the final PDF page sequence. Chapter names and status markers are source-grounded; derived counts are explicitly labeled in their figure notes.

1. Introduction and project context

Why EMS exists, what it covers, and the boundaries of this documentation.

1.1 Project overview

The Employee Management System (EMS) is a multi-tenant human capital and workforce operations platform for organizations that need a single operational view of employees, attendance, leave, payroll, recruitment, onboarding, offboarding, assets, documents, notices, and compliance history. The repository describes a TypeScript pnpm monorepo with a Next.js 15 web application, Expo mobile foundation, Tauri desktop foundation, shared packages, and a Supabase PostgreSQL data layer.

1.2 Background and motivation

The project documentation identifies fragmented paper ledgers, spreadsheets, email approvals, and disconnected recruitment and payroll tools as the motivating problem. These processes duplicate data, make approvals difficult to trace, expose sensitive salary information too broadly, and do not provide database-enforced tenant isolation. EMS centralizes the lifecycle behind role-aware portals and relational transactions.

1.3 Problem statement

- Manual and spreadsheet workflows lack row-level tenant isolation and consistent role boundaries.
- Attendance, leave, recruitment, onboarding, payroll, and offboarding are often handed off manually.
- Concurrent edits can overwrite history and formula-driven calculations can be tampered with.
- Sensitive changes require immutable audit trails for operational and compliance review.

1.4 Objectives

- Provide a secure multi-tenant platform with PostgreSQL Row Level Security (RLS).
- Expose ten specialized portals governed by a canonical least-privilege permission matrix.
- Automate the employee lifecycle from candidate handoff through onboarding, daily operations, payroll, and exit.
- Keep validation, types, configuration, and UI primitives reusable across clients.
- Document actual verification status without fabricating runtime metrics or screenshots.

1.5 Scope

Scope area	Included in current repository	Boundary / status
Web operations	Next.js portal, auth, dashboards, CRUD, workflows, server actions	Primary implemented surface; current build gate needs repair.
Shared platform	types, validation, config, UI, shared utilities, feature registry	Implemented packages and cross-app contracts.
Data and security	Supabase Auth, PostgreSQL migrations, RLS, triggers, RPCs, seed data	47 SQL-created tables and 9 migration files derived from code.
Mobile / desktop	Expo and Tauri foundations	Architecture foundation; native binaries not verified.
Future integrations	Biometric bridge, live banking APIs, AI analytics, localization	Pending/future scope; not claimed as delivered.

2. Existing system versus proposed EMS

The comparison is grounded in project-report.md and requirements-specification.md.

2.1 Existing process pattern

The documented existing pattern uses paper registers or punch cards for attendance, physical leave slips, Excel payroll sheets, email-based recruitment, and shared files for employee documents. It is a general problem statement rather than a survey of one named organization.

2.2 Proposed EMS model

EMS introduces a centralized relational data model, server-side action boundaries, RLS policies, transactional RPCs, immutable history tables, and ten portals. The UI is intended to make the right workflow visible, while the server and database remain the authorization boundary.

Dimension	Existing manual / spreadsheet process	Proposed EMS v1.0.0	Evidence status
Data architecture	Multiple files and physical registers	Central PostgreSQL schema with organization_id scoping	Documented + SQL-derived
Access control	File sharing; broad visibility	Portal roles, 22 permissions, server checks, RLS	Config + migration + tests
Approvals	Paper, email, verbal handoffs	Status records and transactional approval RPCs	Workflow docs + SQL
Attendance to payroll	Manual reconciliation	Work sessions, leave ledger, payroll calculation inputs	Implemented code paths; live integration gate separate
Auditability	No reliable history	employee_history, task_activity_log, approval history	Schema/documented
Export safety	Formula injection risk	CSV neutralization for formula prefix characters	Shared utility + tests
Client strategy	Desktop files and email	Web implementation plus mobile/desktop foundations	Web current; native foundation

Interpretation

The proposed platform improves control through database constraints and traceable workflow state. It does not eliminate the need for organization-specific policy configuration, operational monitoring, rate limiting, backups, or live acceptance testing.

3. Software requirements specification

Functional and non-functional requirements consolidated from requirements-specification.md.

3.1 Functional requirements

ID	Area	Requirement summary
FR-AUTH	Authentication and session	Verified email/password, HttpOnly cookies, recovery, route redirects, open-redirect defense.
FR-ORG	Organization and hierarchy	Multiple organizations, branches, hierarchical departments, designations, cross-org assignment prevention.
FR-EMP	Employee lifecycle	Directory search/filter/pagination, CRUD, unique codes, immutable history, safe CSV export.
FR-ATT	Attendance and shifts	Clock in/out, work sessions, breaks, overtime, overlap prevention, monthly history.
FR-LEV	Leave management	Leave types, balances, date/reason requests, quota guards, manager approvals and ledger updates.
FR-PAY	Payroll engine	Salary structures, duplicate prevention, sequential state machine, immutable approved records, payslips.
FR-REC	Recruitment and ATS	Vacancies, pipeline stages, interviews, scorecards, offers, candidate-to-employee handoff.
FR-OPS	Operations and talent	Onboarding/offboarding guards, assets, appraisals, documents, announcements and read receipts.

3.2 Non-functional requirements

Category	Requirement	Verification / caveat
Security	RLS enabled on operational tables; triggers block self-escalation; Zod validates inputs; Auth hashes passwords.	SQL and source support the design; database pgTAP execution is pending Docker per status tracker.
Reliability	ACID transactions for critical state changes; append-only audit trails.	RPCs/triggers are present; production failure recovery remains an operational concern.
Performance	Sub-500ms initial rendered response target; indexed lookups.	Target is documented; no load measurement was supplied.
Portability	pnpm monorepo; web on modern browsers; shared contracts for mobile/desktop.	Web route source is present; native packaging is foundation-level.
Maintainability	Strict TypeScript, Zod schemas, migrations, reusable UI components.	Current typecheck/lint are not clean on the working tree; see Chapter 14.

3.3 Platform assumptions

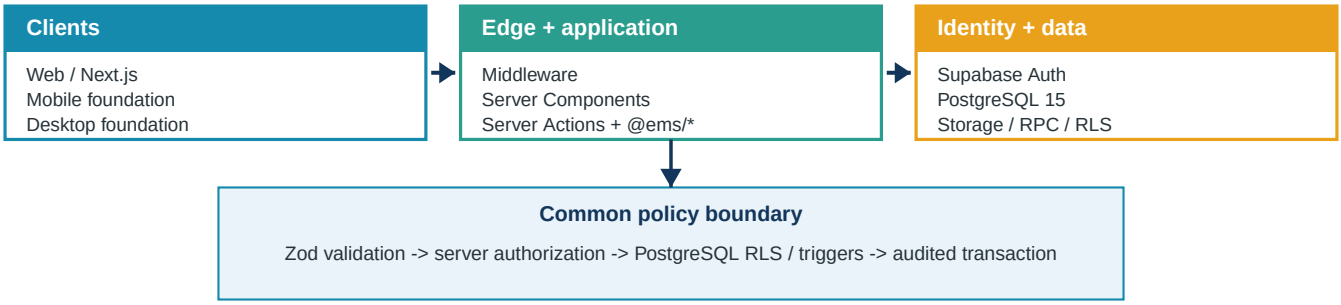
- Node.js 20.x or 22.x LTS is documented; repository README records Node.js v26.5.0 in use for the current environment.
- pnpm 9.x, TypeScript 5.5+, Supabase PostgreSQL 15, Docker for local Supabase, and modern browsers are documented prerequisites.
- Organization-specific payroll, leave, tax, biometric, backup, and rate-limit policies remain configurable dependencies.

4. Current architecture and monorepo topology

Current repository topology, technology boundaries, and implementation status.

4.1 Three-tier architecture

Figure: Current 3-tier request architecture (redrawn from docs/architecture.md)



4.2 Monorepo topology

Area	Path / package	Responsibility	Status
Application	apps/web	Next.js 15 App Router; React 19; Tailwind; middleware; portals; data actions.	Implemented web surface; current build gate fails.
Application foundation	apps/mobile	Expo SDK 52 / React Native shared-contract foundation.	Architecture foundation.
Application foundation	apps/desktop	Tauri v2 / React Windows foundation.	Architecture foundation.
Shared contract	packages/types	Canonical roles, permissions, domain interfaces and aliases.	Implemented.
Shared validation	packages/validation	Zod schemas for auth, employee, attendance, leave, payroll, ATS, operations.	Implemented; recent auth changes are in working tree.
Shared configuration	packages/config	Institution metadata, ten role definitions, routes, navigation, permission matrix, evaluators.	Implemented; source used for RBAC figure.
Shared design system	packages/ui	Institutional metadata, light/dark tokens, spacing, radii, UI data interfaces.	Implemented package contracts.
Shared utilities	packages/shared	Formatting, redirect safety, CSV sanitization and helpers.	Implemented.
Feature flags	packages/feature-registry	Environment-aware feature registry.	Foundation / shared package.
Persistence	supabase/migrations + seed.sql	PostgreSQL schema, RLS, triggers, RPCs, deterministic seed.	47 tables derived; pgTAP execution pending.

4.3 Web application structure

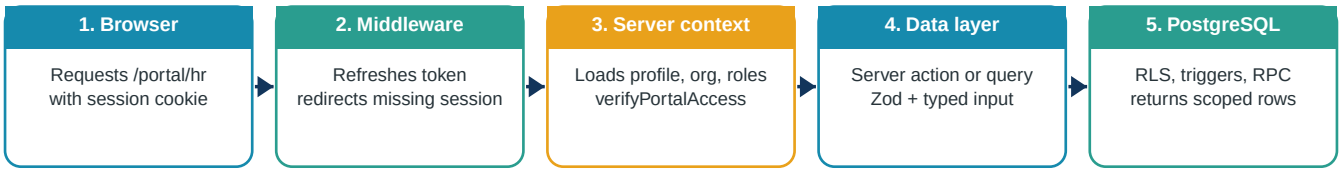
```
apps/web/
  app/                route pages, auth, portal shells and layouts
  components/         client dashboards, tables, widgets and UI
  lib/auth/            session, access and server auth helpers
  lib/data/            typed domain data access and server actions
  middleware.ts        session refresh, cookie sync and route protection
```

The route tree currently contains ${routes.length} page.tsx files (derived source inventory, including public/auth/support routes). This is not the same as a successful production build route count.

5. Request, authentication and data flow

The security path from browser request to tenant-scoped database response.

Figure: protected request path



Derived from docs/architecture.md, docs/AUTH_AND_RBAC.md, apps/web/lib/auth, and apps/web/lib/data.

5.1 Login and session sequence

1. The user submits email and password to the login action; validation occurs before Supabase Auth.
2. Supabase Auth returns a session; the SSR integration persists the session in secure, HttpOnly cookies.
3. Next.js middleware refreshes and validates the session for protected portal requests.
4. The server loads the profile, organization membership, active role assignments, and permissions.
5. The portal gate checks account state, active organization, role/permission, and requested destination.
6. The data module executes a typed query or server action; PostgreSQL applies RLS and integrity triggers.
7. Mutations revalidate the route and append the applicable history/audit record where designed.

5.2 Authorization boundaries

Boundary	Control	Failure path
Client UI	Permission-filtered navigation and disabled controls	Usability only; never treated as final authorization.
Next.js middleware	Session presence, token refresh, auth redirect	Redirect to /login with internal redirect target.
Server context / gate	Active account, organization, role and permission resolution	Redirect to /unauthorized or /access-pending.
PostgreSQL RLS	organization_id membership and permission predicates	Rows are filtered or mutation is rejected.
Triggers / RPCs	Cross-org boundaries, state machines, idempotency, immutable locks	Transaction raises a descriptive exception.

Current code verification note

The working tree contains a recent auth/access refactor. The current typecheck and build fail because getCurrentUserContext returns a nullable activeOrganization where ActiveOrganizationContext requires a non-null Organization. This is documented as an observed gate result, not silently converted to a pass.

6. Database design and entity inventory

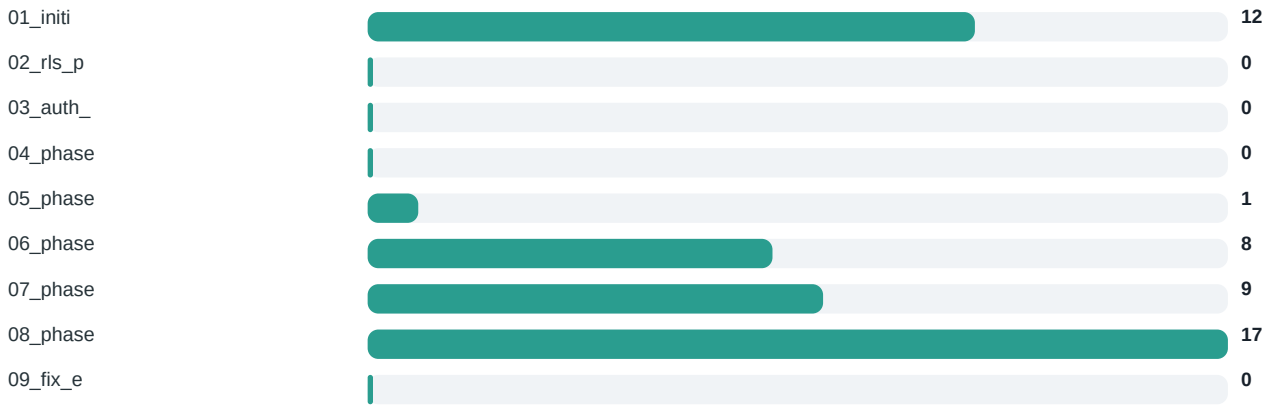
Schema structure derived from current SQL migrations, with documented design interpretation.

6.1 Database standards

- PostgreSQL 15 via Supabase; UUID primary keys generated with gen_random_uuid() in the core schema.
- TIMESTAMPTZ timestamps and set_updated_at() triggers on mutable tables.
- Tenant-owned records carry organization_id and use foreign keys with restrictive or controlled delete behavior.
- RLS policies, SECURITY DEFINER helpers, cross-organization triggers, and transactional RPCs protect the data boundary.

6.2 Migration and table coverage

Figure: SQL-created tables by current migration file



Derived from CREATE TABLE IF NOT EXISTS statements in supabase/migrations. Migrations 000002-000004 and 000009 add policies/functions/hotfixes without new tables.

Migration	Role in repository	Tables created	Policy statements
00001_initial_schema.sql	Core schema	12	0
00002_rls_policies.sql	Baseline RLS policies	0	38
00003_auth_triggers.sql	Auth triggers	0	0
00004_phase3_rbac_portals.sql	Phase 3 RBAC portals	0	1
00005_phase5_employee_management.sql	Phase 5 employee extension	1	14
00006_phase6_attendance_shifts_leave.sql	Phase 6 attendance, shifts, leave	8	19
00007_phase7_payroll_and_recruitment.sql	Phase 7 payroll and recruitment	9	25
00008_phase8_operations_and_talent.sql	Phase 8 operations and talent	17	34
00009_fix_employees_rls_recursion.sql	Phase 9 RLS recursion hotfix	0	1

Current source-derived total: 47 SQL-created application tables across 9 migration files and 132 CREATE POLICY statements. The authored docs' 8-migration / 35+ table summary is retained as a documented baseline and flagged for reconciliation.

6.3 Entity inventory by domain

Domain	Entities (current SQL names)
Core / identity	organizations; branches; departments; designations; profiles; employees; employee_history; roles; permissions; role_permissions; user_roles; settings; employee_code_sequences
Attendance / leave	shifts; shift_assignments; attendance_records; attendance_corrections; leave_types; leave_requests; leave_ledger; notifications

Domain	Entities (current SQL names)
Payroll / compensation	salary_structures; payroll_runs; payroll_records; payroll_approval_history
Recruitment	job_postings; candidates; candidate_stage_history; interviews; job_offers
Operations / talent	onboarding_checklist_templates; onboarding_cases; onboarding_tasks; offboarding_cases; offboarding_clearance_items; performance_cycles; performance_goals; performance_reviews; employee_tasks; task_activity_log
Assets / documents / communication	assets; asset_assignments; asset_maintenance; documents; announcements; announcement_reads; shared_approvals

6.4 Referential integrity and trigger controls

Control	Purpose
enforce_employee_org_boundary()	Branch, department, and designation assigned to an employee must belong to the same organization.
enforce_department_branch_org_boundary()	A department branch must belong to the department organization.
enforce_user_role_org_scope()	A role must be a system role or belong to the user_roles organization.
prevent_self_org_reassignment()	Blocks a user from changing their own organization membership.
prevent_self_role_assignment()	Blocks self-assignment of elevated roles.
Payroll / lifecycle RPCs	Make approval, idempotency, immutable locking, onboarding, offboarding, and asset constraints transactional.

7. API and server operations

EMS primarily exposes authenticated Next.js Server Actions and typed data modules rather than public unauthenticated REST endpoints.

API model

The documented API surface is an internal server-operation contract. Actions authenticate through the HTTP cookie session, validate input with shared Zod schemas, call typed data modules, and rely on Supabase/PostgreSQL authorization. Public REST endpoint claims should not be inferred from the Server Action list.

7.1 Server Action summary

Domain	Operations documented in repository	Primary permissions / boundary
Auth	signInAction; signOutAction; requestPasswordResetAction; updatePasswordAction	Unauthenticated entry for sign-in/recovery; session required for update/sign-out.
Employees	getEmployees; getEmployeeById; createEmployeeAction; updateEmployeeAction	employee.read / employee.write; employee_history on sensitive changes.
Attendance	clockInAction; clockOutAction; getMonthlyAttendance	attendance.self for employee; manager/HR scopes for rosters and corrections.
Leave	submitLeaveRequestAction; processLeaveApprovalAction	leave.self; leave.approve; authoritative leave ledger.
Payroll	generatePayrollRunAction; transitionPayrollStatusAction; getPayslipById	payroll.read/write/approve; duplicate period constraint and state machine.
Recruitment	job posting, candidate stage, interview, offer, handoff operations	recruitment.read/write; stage transition and idempotent handoff RPCs.
Operations	completeOnboardingCaseAction; processOffboardingDecisionAction; asset assign/return; submitPerformanceReviewAction	employee.write; asset.manage; rating and clearance guards.

7.2 Database RPC signatures

```
SELECT public.handoff_candidate_to_employee(p_application_id, p_employee_code, p_joining_date);
SELECT public.complete_onboarding_case(p_case_id);
SELECT public.process_offboarding_decision(p_case_id, p_decision);
SELECT public.assign_asset(p_asset_id, p_employee_id, p_notes);
SELECT public.return_asset(p_assignment_id, p_condition);
```

7.3 Common result and error patterns

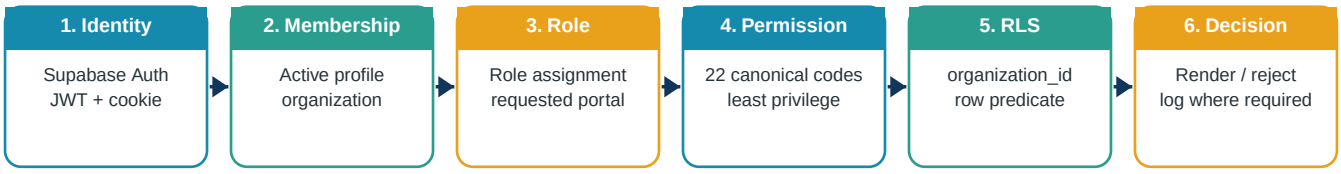
- Invalid input: rejected by Zod before database mutation.
- Unauthenticated session: redirected to /login or action returns a session error.
- Insufficient role/permission: portal gate redirects to /unauthorized; RLS remains authoritative.
- Illegal workflow state: RPC or server action rejects the transition with a descriptive error.
- Cross-tenant reference: database trigger rejects the insert/update.

8. Security, RLS and threat controls

Defense-in-depth controls across browser, server, database and exports.

8.1 RBAC and RLS flow

Figure: authorization decision path



Conceptual diagram redrawn from docs/AUTH_AND_RBAC.md and security-design.md; the UI is not the security boundary.

8.2 Threat-control inventory

Threat / failure mode	Mitigation in repository	Evidence / status
Cross-tenant read/write	RLS policies using is_org_member(organization_id); scoped queries; boundary triggers	38 baseline policies plus later phase policies derived; pgTAP execution pending.
Privilege self-escalation	prevent_self_org_reassignment and prevent_self_role_assignment triggers	Migration and security tests documented; current Vitest suite passes relevant cases.
Search-path hijacking	SECURITY DEFINER functions declare SET search_path = ''	SQL source and security-design.md.
Open redirects	sanitizeRedirectUrl rejects absolute, protocol-relative and backslash paths	Source and current security tests.
CSV formula injection	sanitizeCsv / escapeCsvField prefixes =, +, -, @, tab, CR	Shared utility and attendance/payroll/ operations tests.
Session token exposure	@supabase/ssr; HttpOnly, Secure, SameSite cookie design	Architecture/security docs; live browser verification not asserted here.
Illegal payroll mutation	Sequential transitions and immutable approved records	Phase 7 SQL, actions, tests.
Asset double assignment	assign_asset enforces one active assignment and availability state	Phase 8 SQL, actions, tests.
Incomplete lifecycle closure	Onboarding required task guard; offboarding clearance guard	Phase 8 SQL, actions, tests.

8.3 Production risks still requiring operational controls

- Enable rate limiting on login and recovery routes in the deployment edge or API gateway.
- Use a private storage policy and short-lived signed URLs if document storage is exposed in production.
- Rehearse backup and restore in a non-production environment and record recovery evidence.
- Use mTLS or equivalent controls for any future biometric gateway.

9. Roles, portals and permission coverage

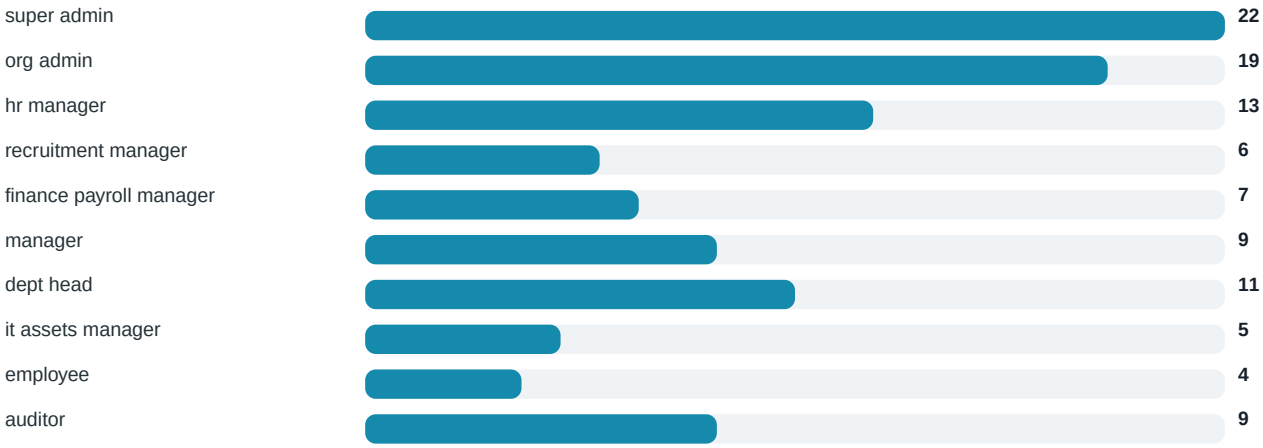
Ten role definitions and route mappings are sourced from @ems/config and the RBAC documents.

9.1 Portal inventory

Portal	Base path	Role code	Primary responsibility	Status
Super Admin	/portal/super-admin	super_admin	Tenant provisioning, global configuration, audit	Implemented
Organization Admin	/portal/org-admin	org_admin	Branches, departments, designations, policies, members	Implemented
HR Manager	/portal/hr	hr_manager	Employee directory, attendance, leave, lifecycle, docs	Implemented
Recruitment	/portal/recruitment	recruitment_manager	Openings, candidates, interviews, offers, handoff	Implemented
Finance & Payroll	/portal/finance-payroll	finance_payroll_manager	Salary structures, runs, approvals, reports	Implemented
Manager	/portal/manager	manager	Direct reports, attendance, leave approvals, tasks	Implemented
Department Head	/portal/dept-head	dept_head	Department roster, leave, assets	Implemented
IT & Assets	/portal/it-assets	it_assets_manager	Inventory, allocations, maintenance	Implemented
Employee Self-Service	/portal/employee	employee	Attendance, leave, payslips, docs, notices	Implemented
Compliance Auditor	/portal/auditor	auditor	Read-only history, payroll and security audit	Implemented

9.2 Role count and permission coverage

Permissions configured per role



Derived from packages/config/src/index.ts. Canonical permission catalog contains 22 codes; counts indicate configured capabilities, not executed requests.

Role	Permissions	Coverage note
super_admin	22	Full configured catalog
org_admin	19	Least-privilege domain subset
hr_manager	13	Least-privilege domain subset
recruitment_manager	6	Least-privilege domain subset

Role	Permissions	Coverage note
finance_payroll_manager	7	Least-privilege domain subset
manager	9	Least-privilege domain subset
dept_head	11	Least-privilege domain subset
it_assets_manager	5	Least-privilege domain subset
employee	4	Self-service only
auditor	9	Read-only inspection boundary

9.3 Role behavior summary

- Super Admin can access platform-wide portal routes and has all configured permission codes.
- Organization Admin manages tenant structure and role/security configuration within the organization.
- HR owns employee records and lifecycle operations; Recruitment owns the ATS; Finance owns payroll.
- Manager and Department Head receive scoped supervisory/department capabilities rather than payroll administration.
- IT & Assets manages hardware; Employee is self-service; Auditor is read-only by design and test coverage.

9A. Full RBAC Permission Matrix

Source: packages/config/src/index.ts ROLE_PERMISSIONS_MATRIX, cross-checked against packages/types/src/index.ts. YES means the permission is in the role's documented configuration. The matrix is a capability configuration, not a runtime access log.

Permission	Super Admin	Org Admin	Hr Manager	Recruitment Manager	Finance Payroll Manager	Manager	Dept Head	It Assets Manager	Employee	Auditor
organization.read	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES
organization.write	YES	YES	-	-	-	-	-	-	-	-
employee.read	YES	YES	YES	YES	YES	YES	YES	YES	-	YES
employee.write	YES	YES	YES	-	-	-	-	-	-	-
attendance.self	YES	YES	YES	-	-	YES	YES	-	YES	-
attendance.read	YES	YES	YES	-	-	YES	YES	-	-	YES
attendance.manage	YES	YES	YES	-	-	YES	YES	-	-	-
leave.self	YES	YES	YES	-	-	YES	YES	-	YES	-
leave.read	YES	YES	YES	-	-	YES	YES	-	-	YES
leave.approve	YES	YES	YES	-	-	YES	YES	-	-	-
payroll.read	YES	YES	-	-	YES	-	-	-	-	YES
payroll.write	YES	-	-	-	YES	-	-	-	-	-
payroll.approve	YES	-	-	-	YES	-	-	-	-	-
recruitment.read	YES	YES	YES	YES	-	-	-	-	-	-
recruitment.write	YES	-	-	YES	-	-	-	-	-	-
asset.read	YES	YES	-	-	-	-	YES	YES	-	YES
asset.manage	YES	YES	-	-	-	-	-	YES	-	-
document.read	YES	YES	YES	YES	YES	-	YES	-	YES	YES
document.write	YES	YES	YES	-	-	-	-	-	-	-
report.read	YES	YES	YES	YES	YES	YES	YES	YES	-	YES
audit.read	YES	YES	-	-	-	-	-	-	-	YES
role.manage	YES	YES	-	-	-	-	-	-	-	-

Coverage summary: 10 roles x 22 permissions; Super Admin has 22; Employee has 4; Auditor has 9.

10. Feature inventory and implementation status

Status is reconciled against code paths, SQL migrations, tests, and documentation claims.

Feature / module	Status	Repository evidence and verification boundary
Authentication, session cookies, recovery	Implemented	Routes and actions in apps/web; Supabase SSR and middleware documented; current typecheck has a session-context error.
Organization, branches, departments, designations	Implemented	Core schema, admin routes, config navigation, and organization RLS are present.
Employee directory and CRUD	Implemented	HR directory, create/edit/detail routes, validation, history trigger, and CSV export are present.
Attendance, shifts, corrections	Implemented	Phase 6 migration, actions, RPCs, HR/manager/employee screens, and 17 tests are present.
Leave balances and approvals	Implemented	Ledger-based balance calculations, queues, approval RPC, and validation are present.
Payroll and payslips	Implemented	Phase 7 schema/actions/UI and 12 tests are present; bank transfer API is not integrated.
Recruitment and candidate handoff	Implemented	Pipeline, interviews, offers, idempotent handoff, and connected UI are present.
Onboarding and offboarding	Implemented	Checklist/clearance guards, transactional RPCs, HR screens, and tests are present.
IT assets and maintenance	Implemented	Inventory, assignment/return constraints, maintenance route, and tests are present.
Performance, tasks, documents, announcements	Implemented	Phase 8 tables/actions/clients and HR/employee routes are present.
Auditor read-only portal	Implemented	Portal and permission tests document the read-only boundary.
Mobile application	Architecture foundation	apps/mobile exists and shares packages; native binary build is not verified.
Desktop application	Architecture foundation	apps/desktop exists and shares packages; native installer build is not verified.
Database pgTAP suite	Needs verification	60 assertions are documented; DEVELOPMENT_STATUS says execution is pending Docker.
Live performance / load targets	Needs verification	Sub-500ms is a requirement target, not a measured runtime metric.
Biometric IoT bridge and direct banking APIs	Pending / future	Explicitly listed as future scope in limitations-and-future-scope.md.

10.1 Status interpretation

Implemented does not mean live acceptance

A module is marked Implemented when the current repository includes its schema/actions/routes or tests and the documentation describes it as delivered. Runtime deployment, external integrations, native packaging, database execution, and production monitoring remain separate evidence gates.

10.2 Source-derived UI surface

Route family	Current route pages (derived)	Representative screens
Public/auth	about, login, signup, forgot-password, auth/reset-password, unauthorized, /	Landing, sign-in, recovery, reset, access denied.
Super Admin	5	Overview, organizations, roles, audit, settings.
Organization Admin	7	Overview, branches, departments, members, policies, roles, audit.

Route family	Current route pages (derived)	Representative screens
HR	17	Directory, detail/edit/create, attendance, shifts, leave, lifecycle, performance, docs, announcements.
Recruitment	4	Overview, openings, candidate pipeline, onboarding.
Finance & Payroll	5	Overview, structures, runs, approvals, reports.
Manager / Dept Head	8	Team, approvals, shifts, staff, leave, assets.
IT & Assets	4	Overview, inventory, allocations, maintenance.
Employee	5	Dashboard, attendance, leave, payslips, documents.
Auditor	4	Overview, logs, changes, payroll audit.

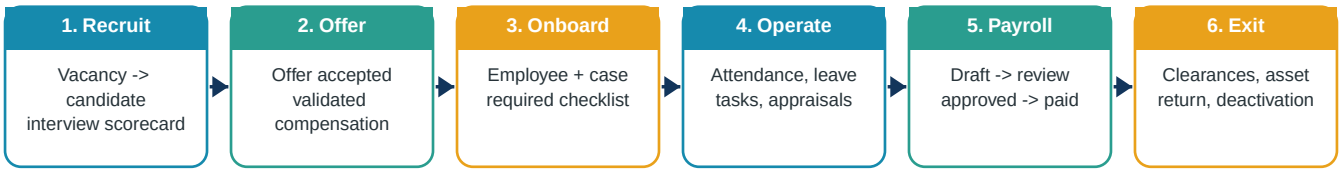
Source inventory count: 67 page.tsx files under apps/web/app. The earlier authored docs state 34 compiled routes; current build did not complete, so no compiled route count is claimed here.

11. Module workflows and algorithms

Business rules are summarized from algorithms-and-workflows.md, module-documentation.md, API docs, and phase SQL.

11.1 Employee lifecycle workflow

Figure: recruitment-to-offboarding lifecycle



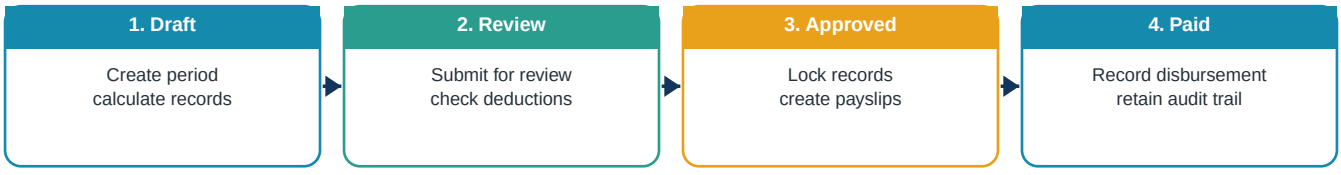
Derived workflow; individual transitions are guarded by validation, RPCs, constraints, and RLS as described in the repository.

11.2 Core algorithms

Algorithm / guard	Rule	Failure prevention
Portal authorization	Require user, active profile, active organization and required role/ permission.	Unauthenticated or unauthorized navigation is redirected/rejected.
Attendance duration	Net minutes = max(0, clockOut - clockIn - breakMinutes); standard hours capped at 8; remainder is overtime.	Prevents negative totals; handles overnight sessions in phase logic.
Leave balance	Available balance is derived from ledger usage and pending reservations; request cannot exceed availability.	Prevents quota exhaustion and inconsistent mutable balance.
Payroll net pay	Basic less loss-of-pay plus allowances; statutory deductions and tax are subtracted; monetary values round to two decimals.	Duplicate period and illegal status transitions are rejected.
Onboarding closure	All required tasks must be completed before case closure.	Incomplete joiner checklist blocks activation/ closure.
Offboarding closure	All IT, Finance, HR and other required clearance items must be cleared before deactivation.	Pending return/settlement blocks final exit.
Asset assignment	Asset must be available and have no active assignment; retired/ maintenance assets are blocked.	One active assignment per device.
CSV sanitization	Prefix cells beginning =, +, -, @, tab, or carriage return with a single quote.	Spreadsheet applications treat values as text.

11.3 Payroll state machine

Figure: payroll approval lifecycle



The exact code uses pending_approval in the Phase 7 RPC description in DEVELOPMENT_STATUS; authored docs also use review. Confirm the final state label used in the deployed schema.

12. UI/UX and screen inventory

Screens are inventoried from route files, config navigation, and design-system documentation; no live screenshots are asserted.

12.1 Design system

Token / primitive	Current repository definition
Light palette	Primary #1e3a8a; secondary #0284c7; accent #f59e0b; background #f8fafc; success #10b981; warning #f59e0b; danger #ef4444.
Dark palette	Primary #3b82f6; secondary #38bdf8; accent #fbbf24; background #0f172a; card #1e293b; status colors remain semantic.
Layout	Responsive AppShell; collapsible desktop sidebar 260px <-> 72px; mobile drawer; header with breadcrumbs, role/org switchers and notifications.
Primitives	Button, Card, Badge, MetricCard, Input, Select, DataTable, SearchFilterBar, Modal, ConfirmDialog, Toast, Skeleton, Breadcrumbs, Pagination, Empty/Error states.
Accessibility intent	Labeled inputs, focus rings, modal Escape dismissal/focus trap, keyboard-aware controls, chart table views.

12.2 Screen inventory

Portal / route group	Screen inventory	UX behaviors
Public and authentication	Landing, login, signup, forgot-password, reset-password, callback, unauthorized/not-found.	Inline validation, password visibility, redirect safety, institutional branding.
Super Admin	Overview; organizations; roles; audit; settings.	Tenant table, system roles, audit inspection, settings surfaces.
Org Admin	Overview; departments; branches; members; policies; roles; audit.	Hierarchy management, member and policy tables, audit history.
HR	Overview; employees list/new/detail/edit; branches; departments; designations; attendance; shifts; leave; onboarding; offboarding; performance; documents; announcements; import/export.	Search/filter/pagination, modals, progress/ clearance states, CSV export.
Recruitment	Overview; openings; candidates; onboarding.	Stage tabs, inspector modal, interview scheduler, feedback, offer and handoff actions.
Finance	Overview; salary structures; payroll runs; approvals; reports.	KPI cards, run status actions, audit history, payslip inspection and CSV export.
Manager / Dept Head	Team, approvals, shifts, staff, leave, assets.	Scoped rosters, approval queues, reporting-line actions.
IT & Assets	Overview; inventory; allocations; maintenance.	Inventory table, assignment/return modals, condition and maintenance logs.
Employee	Dashboard; attendance; leave; payslips; documents.	Clock-in/out widget, balance cards, leave request, digital payslip view.
Auditor	Overview; logs; changes; payroll audit.	Read-only inspection; mutation controls intentionally absent.

Screenshot evidence

The repository includes a screenshot guide and route mappings, but this submission PDF contains no claimed live screenshots. Any future capture should be labeled as an actual build screenshot and dated with the environment used.

13. Team, lead and module ownership

The current canonical assignment is taken from project-report.md and review-presentation-outline.md.

13.1 Team roster

Member	Role / documented ownership
Teja (Team Lead)	Monorepo architecture; multi-tenant RLS; system integration; security auditing.
Ashok	Database migrations; PostgreSQL entity relationships; cross-organization triggers.
Charan	Supabase Auth integration; session management; RBAC matrix; portal access routing.
Kula Shekhar	Attendance clock-in/out engine; shift policies; leave approval workflows.
Jahnavi	Payroll engine; salary structures; sequential run state machines; payslip generation.
Afsar	Operations modules: onboarding, offboarding, IT assets, documents, announcements; automated test suites.

Figure: documented responsibility-topic coverage



Derived by counting the responsibility topics listed in the current canonical assignment. This is coverage breadth, not contribution effort, code volume, or quality.

13.2 Assignment inconsistency to confirm

Topic	Current canonical wording	Earlier conflicting wording / action
Team ownership	Current canonical docs (project-report.md and review-presentation-outline.md) assign Charan to Auth/RBAC, Kula to Attendance/Leave, Jahnavi to Payroll, and Afsar to Operations/Testing.	Earlier embedded artifact `.opencode/omnirush/inbox/.../text.md` assigns Ashok to Attendance/Leave, Charan to Payroll/Reports, Kula to UI/UX/PowerPoint/diagrams, Jahnavi to Recruitment/ Onboarding/Performance, and Afsar to SRS/Database/API/User Manual. Confirm the authoritative assignment before submission.
Migration count	Most authored docs describe 8 progressive migrations and 35+ tables.	The current repository contains 9 SQL migrations and 47 CREATE TABLE statements; migration 000009 is a Phase 9 RLS recursion hotfix. Use 9/47 for code-derived coverage and retain 8/35+ as the earlier documented baseline.
Verification status	project-report, release notes, and testing-report claim 100/100 tests, clean typecheck/lint, and 34 compiled routes.	Current observed run on the working tree: 103/103 Vitest tests pass; typecheck and build fail on `AuthenticatedUserContext.activeOrganization`; lint fails on boolean comparisons in auth files. The observed state is recorded in Chapter 14.
Phase marker	DEVELOPMENT_STATUS reaches Phase 10 complete.	README.md still says current phase is Phase 3. Confirm which phase label should be used in the final academic cover sheet.

Submission action

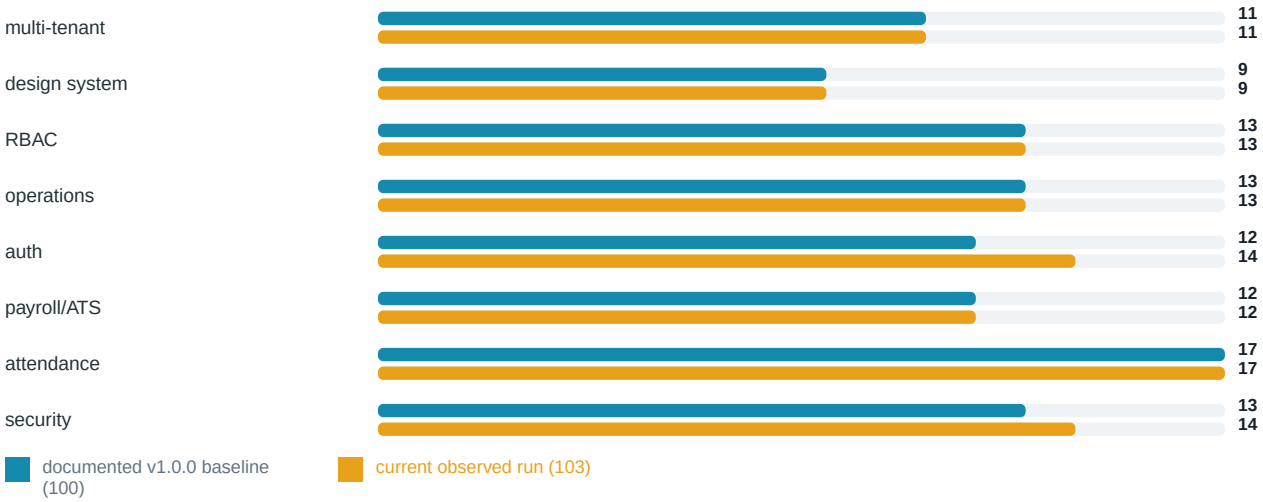
Before a final academic submission, confirm the team responsibility sheet with Team T-1. Keep one authoritative assignment table and update the earlier embedded artifact or clearly mark it superseded.

14. Testing and actual-versus-planned status

The authored release baseline and the current working-tree verification are shown separately.

14.1 Current observed test suites

Test counts by suite: documented baseline versus current observed source



Suite	Current source-derived	Documented baseline	Current observed status
multi-tenant	11	11	PASS
design system	9	9	PASS
RBAC	13	13	PASS
operations	13	13	PASS
auth	14	12	PASS
payroll/ATS	12	12	PASS
attendance	17	17	PASS
security	14	13	PASS

The 103 count is consistent with the current test files and the observed `pnpm test` run: auth is 14 and security is 14, whereas the authored report lists 12 and 13. All 8 suites passed in the observed run.

14.2 Quality gates

Gate	Documented release claim	Current observed result	Status
Vitest	100/100; 8 suites	103/103; 8 suites	PASS observed
TypeScript	0 errors across 9 workspaces	apps/web session context type error	FAIL observed
ESLint	0 errors / 0 warnings	3 auth files with boolean-compare errors; unused-disable warnings	FAIL observed
Web build	34 routes compiled	Compilation succeeded, type validation failed	FAIL observed
Database pgTAP	60 assertions documented	Not executed; Docker/Supabase requirement remains	PENDING

14.3 Planned or unverified test coverage

- Database integration execution against a local Supabase/Docker stack.
- Native Android APK/AAB and Windows installer packaging.

- Performance/load testing against the sub-500ms requirement target.
- Production backup/restore rehearsal, rate-limit behavior, and document storage acceptance.
- Live browser smoke tests for each portal using seeded accounts; repository screenshot guide is a procedure, not execution evidence.

15. Installation and deployment

Operational runbook consolidated from installation-and-setup.md, deployment-guide.md, README.md and package scripts.

15.1 Prerequisites

Tool	Documented requirement	Current repository note
Node.js	20.x or 22.x LTS	README records v26.5.0 in use for this environment.
pnpm	9.x or higher	Repository packageManager is pnpm@9.15.9.
Git	Recent version	Required for clone and release workflow.
Docker Desktop	Required for local Supabase	Database tests and reset depend on availability.
Supabase CLI	Local or hosted database management	Migration, seed, status and pgTAP commands are provided.
Rust / Android Studio	Desktop/mobile builds only	Native packaging is foundation-level and unverified.

15.2 Local setup

```
pnpm install
copy .env.example .env.local
# configure Supabase URL and anon key; never commit secrets
pnpm db:start       # Docker + local Supabase
pnpm db:reset       # destructive local reset: migrations + seed
pnpm dev:web        # Next.js development server
```

15.3 Verification commands

```
pnpm test
pnpm typecheck
pnpm lint
pnpm format:check
pnpm --filter @ems/web build
pnpm --filter @ems/web start
```

15.4 Production deployment

1. Use Vercel or AWS ECS/Node.js for apps/web and a managed Supabase project for PostgreSQL/Auth/Storage.

2. Run tests, typecheck, lint, migrations and environment checks in pre-flight; change all seeded development passwords.

3. Set NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, NEXT_PUBLIC_SITE_URL and any server-only secret in the deployment environment.

4. Build with pnpm --filter @ems/web build and start with pnpm --filter @ems/web start; current observed build must be repaired before claiming release readiness.

5. Back up the database, rehearse restore, and keep migration rollback procedures with the deployment record.

Secret handling

The service-role key bypasses RLS and must stay in trusted server/deployment environments. Never put it in web, mobile, desktop client bundles or committed environment files.

16. Limitations and future scope

Explicit boundaries from limitations-and-future-scope.md and current verification evidence.

16.1 Current limitations

Area	Current boundary
Native platform delivery	Mobile and desktop foundations share packages but native APK/AAB and MSI generation require platform-specific toolchains and are not verified here.
Biometric attendance	Web-based geo/IP clocking is documented; direct fingerprint/scanner integration requires an on-premise IoT gateway.
Bank disbursement	Payroll computes slips and exports payment transaction files; live corporate banking API transfers are not integrated.
Measured performance	Sub-500ms is a requirement target; no representative load benchmark is recorded in the repository evidence.
Database execution	The SQL design is present; local pgTAP execution is pending Docker/Supabase availability per status tracker.
Current working-tree gates	Test pass is observed, but typecheck, lint and web build currently fail on recent auth changes.

16.2 Future scope

- AI-driven attrition risk, workforce demand forecasting, and appraisal text analytics.
- MQTT/HTTP biometric bridge with encrypted device authentication and attendance reconciliation.
- Telugu, Hindi, Tamil and other regional localization in config and self-service surfaces.
- Direct banking APIs and automated statutory tax/Provident Fund/ESI return generation.
- Native mobile/desktop release pipelines, offline behavior decisions, observability, rate limits and disaster recovery rehearsal.

17. Conclusion

A concise submission summary with the evidence boundary kept visible.

EMS establishes a coherent architecture for employee and workforce operations: a Next.js web portal surface, shared TypeScript contracts and validation, Supabase Auth, PostgreSQL relational data, RLS tenant boundaries, stateful workflows, and a least-privilege portal model. The repository contains substantial implemented code and SQL across identity, employee management, attendance, leave, payroll, recruitment, operations, assets, documents, announcements and auditing.

The current observed evidence is mixed in a useful way for a technical submission: the Vitest suites pass 103/103 on the current working tree, while typecheck, lint and the production build surface concrete auth refactor issues that should be repaired before a clean release claim is made. The authored v1.0.0 reports record an earlier 100/100 baseline and 34-route build; the discrepancy is documented rather than hidden.

Submission conclusion

The PDF is complete as a repository-grounded technical reference. Before presenting a clean production-readiness statement, reconcile the assignment and migration-count inconsistencies, repair the current auth type/lint failures, execute database tests with Docker/Supabase, and update the release evidence with the final verified commands.

17.1 Source register

Source class	Files used as primary references
Project and status	docs/project-report.md; docs/abstract.md; docs/DEVELOPMENT_STATUS.md; docs/RELEASE_NOTES_v1.0.0.md
Requirements and design	docs/requirements-specification.md; docs/system-design.md; docs/architecture.md; docs/DESIGN_SYSTEM_AND_DASHBOARDS.md
Data and security	docs/database-design.md; docs/security-design.md; docs/AUTH_AND_RBAC.md; supabase/migrations/*.sql
Modules and operations	docs/module-documentation.md; docs/algorithms-and-workflows.md; docs/api-documentation.md
Usage and release	docs/user-manual.md; docs/administrator-manual.md; docs/installation-and-setup.md; docs/deployment-guide.md
QA and future	docs/testing-report.md; docs/limitations-and-future-scope.md; docs/viva-questions.md
Current code anchors	packages/config/src/index.ts; packages/ui/src/index.ts; packages/types/src/index.ts; apps/web/app; apps/web/lib; tests/*.test.ts

18. Viva voce and Q&A appendix

Selected project-specific questions and answers distilled from docs/viva-questions.md, plus current evidence questions.

Question	Answer
What is EMS?	A multi-tenant employee and workforce operations platform covering organization hierarchy, employee records, attendance, leave, payroll, recruitment, operations, assets, documents and audit.
Why not use spreadsheets?	Spreadsheets do not provide database RLS, reliable concurrency, immutable audit history, or a consistent role model.
What is the architecture?	A pnpm monorepo with Next.js web, mobile and desktop foundations, shared packages, Server Actions and Supabase PostgreSQL/Auth.
Why is RLS important?	RLS filters rows inside PostgreSQL so tenant isolation is enforced at the data engine and not only by the UI.
What are the ten roles?	super_admin, org_admin, hr_manager, recruitment_manager, finance_payroll_manager, manager, dept_head, it_assets_manager, employee and auditor.
How many permission codes exist?	The current packages/types source defines 22 canonical codes; packages/config maps them per role.
How is a portal protected?	Middleware validates the session; server context resolves active organization and role; verifyPortalAccess checks role/permission; PostgreSQL RLS remains authoritative.
How is cross-tenant corruption blocked?	Boundary triggers verify branch, department, designation and role ownership against organization_id before writes.
How is self-escalation blocked?	Database triggers block self-organization reassignment and self-role assignment.
How are attendance hours computed?	Elapsed clock-out minus clock-in, less breaks; regular hours are capped at eight and the remainder is overtime in the documented algorithm.
How are leaves controlled?	A request is checked against an authoritative ledger-derived balance and approval atomically writes the decision/debit.
How is duplicate payroll prevented?	The organization and period uniqueness rule and creation action reject duplicate payroll runs.
What is the payroll state machine?	Authored docs describe draft -> review -> approved -> paid; the Phase 7 RPC description also uses pending_approval, so the deployed label should be confirmed.
When do payroll records lock?	The documented design locks approved runs and records as immutable; corrections require a new run.
How does recruitment connect to HR?	An accepted offer invokes a transactional/idempotent handoff that creates an employee and onboarding case.
What blocks onboarding completion?	Any incomplete required checklist task.
What blocks offboarding deactivation?	Any uncleared required department checkpoint, including IT asset return or Finance/HR clearance.
What asset rule is enforced?	One active assignment per asset; retired or under-maintenance assets cannot be assigned.
What is the Auditor boundary?	Read-only inspection of histories, payroll and security records; tests assert no write/manage/approve access.
How is CSV formula injection mitigated?	Cells beginning with =, +, -, @, tab or carriage return are prefixed with a single quote.
Why is SET search_path = '' used?	It reduces search-path hijacking risk in SECURITY DEFINER functions by forcing explicit object resolution.
What did the current test run show?	Eight Vitest suites passed with 103/103 tests on the current working tree.
Did typecheck and build pass in this artifact run?	No. Both fail on the current auth/session activeOrganization nullability mismatch; lint also fails on boolean comparisons in auth files.
Are mobile and desktop complete?	They are architecture foundations sharing types, validation and config; native artifacts were not verified.
Are screenshots included?	No live screenshots are claimed; the UI chapter is a source-derived screen inventory.
What should be fixed before final submission?	Reconcile documentation inconsistencies, repair current quality-gate failures, run pgTAP with Docker, and update evidence with final command outputs.